
PhishX: An Explainable AI-Driven Phishing URL Detection system using SHAP

Usama Rasheed (B-28739)¹, Muhammed Subhan (B-28668)¹ and Taib Ali (Supervisor)¹

¹ *Department of Computer Science, University of South Asia, Lahore, Pakistan*

Reception date of the manuscript: 30/05/2026

Acceptance date of the manuscript: dd/mm/yyyy

Publication date: dd/mm/yyyy

Abstract— Phishing attacks continuously evolve to bypass traditional rule-based security mechanisms, posing a severe threat to global digital infrastructure and leading to multi-billion dollar economic losses annually. While artificial intelligence and deep learning have significantly improved threat detection efficacies, the inherent "black-box" nature of these advanced predictive algorithms limits operational trust among cybersecurity analysts. This research addresses this critical gap by proposing a highly accurate, fully transparent, and deployable phishing URL detection framework. Utilizing the comprehensive PhiUSIIL Phishing URL dataset, we implemented a rigorous preprocessing pipeline. To combat the pervasive issue of class imbalance inherent in cybersecurity data, the Synthetic Minority Over-sampling Technique (SMOTE) was applied. We extensively trained and evaluated six distinct machine learning architectures: Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Artificial Neural Networks (ANN), Random Forests (RF), and eXtreme Gradient Boosting (XGBoost). Empirical results demonstrate that the XGBoost ensemble outperformed all baseline models, achieving a superior accuracy of 98.7% and an F1-Score of 98.7%. To resolve the interpretability challenge, SHapley Additive exPlanations (SHAP), a game-theoretic approach, was integrated to extract global feature importance and local decision logic. The findings reveal that structural URL attributes—such as URL length, subdomain counts, and obfuscation indicators—are the primary drivers of malicious intent. Finally, the proposed architecture was successfully encapsulated into a Flask REST API, offering a robust, real-time deployable solution that balances high predictive capability with essential Explainable AI (XAI) transparency.

Keywords—Artificial Intelligence, Machine Learning, Phishing Detection, Explainable AI (XAI), XGBoost, Cybersecurity, SMOTE, SHAP.

I. INTRODUCTION

The rapid and ubiquitous expansion of digital infrastructure has inadvertently broadened the attack surface for malicious actors. Cybercrime is currently one of the most pressing global challenges, with phishing remaining the primary vector for initial network compromise, credential theft, and ransomware deployment [1]. Phishing involves deploying deceptive communications—most commonly via manipulated Uniform Resource Locators (URLs)—designed to mimic legitimate entities. The Anti-Phishing Working Group (APWG) reported unprecedented volumes of unique phishing sites in recent years, highlighting the inadequacy of legacy defense mechanisms.

Historically, organizations relied heavily on static blacklists, heuristic rules, and signature-based intrusion detection systems. However, these conventional defenses fail against dynamic, zero-day phishing campaigns where

attackers utilize domain generation algorithms (DGAs), URL shorteners, and obfuscation techniques to continuously alter the attack signature [2].

Artificial Intelligence (AI), particularly Machine Learning (ML) and Deep Learning (DL), has emerged as a crucial paradigm shift in modern cybersecurity. ML algorithms are inherently capable of identifying complex, non-linear, and structural patterns within web traffic without relying on pre-existing signatures. Despite achieving near-perfect detection rates in academic literature, a significant operational hurdle prevents the widespread adoption of AI in enterprise Security Information and Event Management (SIEM) systems: the lack of interpretability. Existing AI systems often operate as "black boxes," providing highly accurate predictions but lacking the contextual reasoning required by human security analysts to trust and verify the automated decisions [3].

This research bridges the gap between high predictive accuracy and human-centric interpretability. We propose a comprehensive, end-to-end machine learning pipeline that not only detects phishing URLs with exceptional accuracy but also explains its mathematical reasoning using Explainable AI (XAI) frameworks.

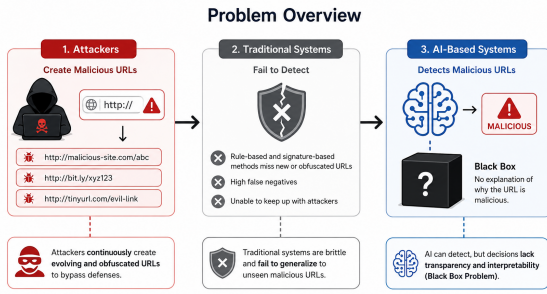


Fig. 1: Problem Overview: The evolution of phishing detection from failing legacy systems to non-interpretable Black-Box AI, highlighting the necessity for Explainable AI.

The primary contributions and objectives of this research are five-fold:

1. To engineer a robust data preprocessing pipeline utilizing the PhiUSIIL dataset, specifically addressing class imbalance through the Synthetic Minority Over-sampling Technique (SMOTE).
2. To systematically train, tune, and evaluate six diverse predictive algorithms: Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Artificial Neural Network (ANN), Random Forest, and XGBoost.
3. To critically analyze the comparative performance of these models using standard evaluation metrics (Accuracy, Precision, Recall, F1-Score).
4. To integrate SHapley Additive exPlanations (SHAP) to dismantle the black-box nature of the best-performing model, extracting both global feature importance and local, instance-level interpretability.
5. To transition the theoretical model into a production-ready environment by deploying it as a live REST API, proving its viability for real-world enterprise integration.

The remainder of this paper is structured as follows: Section 2 provides a comprehensive review of related literature. Section 3 details the mathematical and theoretical framework of the research methodology. Section 4 presents the proposed AI system architecture. Section 5 discusses the empirical results and performance analysis. Section 6 provides an in-depth Explainable AI analysis. Finally, Section 7 concludes the research and outlines future work.

II. LITERATURE REVIEW

The application of artificial intelligence in cybersecurity, specifically for identifying malicious web content, has been extensively documented over the past decade. Early approaches primarily utilized simple lexical analysis. Sahingoz et al. [4] utilized Random Forests combined with Natural Language Processing (NLP) features extracted from URLs, achieving notable accuracy. However, purely lexical models struggle when attackers employ zero-day obfuscation techniques or URL shorteners.

To overcome lexical limitations, researchers transitioned to structural and third-party features. Mohammad et al. [5] introduced a prominent dataset based on 30 distinct URL features, leading to widespread experimentation using Support Vector Machines and Decision Trees. While effective, the dataset size and feature relevance diminished over time due to the evolution of web standards.

Recent advancements have seen the introduction of deep learning. Subasi et al. [6] and Le et al. [7] explored Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks for character-level URL analysis. While these models achieved state-of-the-art accuracy, their computational overhead was massive, and they suffered from the “black-box” dilemma, offering zero explainability to network administrators.

Basit et al. [2] conducted a comprehensive survey on AI-enabled phishing detection, explicitly concluding that while ensemble models like Random Forest and XGBoost perform optimally for tabular data, the field severely lacks interpretable frameworks. Furthermore, Hannousse and Yahiouche [8] highlighted a critical flaw in existing literature: dataset imbalance. Many studies report accuracies above 95% on datasets where 90% of the samples are benign. Without proper balancing techniques, these models suffer from high false-negative rates in real-world scenarios, biasing heavily towards the majority class.

Explainable AI (XAI) has recently gained traction in critical fields like healthcare [9] and finance, but its application in applied cybersecurity remains in its infancy. Our research directly addresses the limitations found in previous studies by pairing an optimized, gradient-boosted ensemble classifier (XGBoost) with a robust class-balancing algorithm (SMOTE), and applying a game-theoretic interpretability framework (SHAP) for complete decision transparency.

III. RESEARCH METHODOLOGY AND THEORETICAL BACKGROUND

This study adopts a quantitative, experimental, and analytical research design. It involves rigorous data processing, mathematical modeling, and empirical evaluation to construct a deployable solution.

a. Data Collection

The dataset utilized is the **PhiUSIIL Phishing URL Dataset** [10], a highly diverse and contemporary collection sourced from the UCI Machine Learning Repository. It contains approximately 235,000 distinct records of benign and malicious URLs. Unlike legacy datasets, PhiUSIIL incorporates continuous numerical features and categorical binary indicators representing structural, lexical, and HTML-based properties of web links (e.g., URLLength, NoOfSubDomain, HasObfuscation).

b. Data Preprocessing and Scaling

Raw network data contains noise, missing values, and varying scales, causing algorithmic instability. The preprocessing pipeline included:

TABLE 1: COMPREHENSIVE LITERATURE REVIEW AND COMPARISON OF EXISTING APPROACHES.

Author (Year)	Algorithm / Technique	Accuracy	Identified Limitations & Research Gaps
Sahingoz et al. (2019) [4]	Random Forest, NLP	97.9%	Ignored zero-day obfuscation techniques; lack of explainability.
Hannousse et al. (2021) [8]	SVM, Web Crawling	95.2%	Severe dataset imbalance led to high false-negative rates.
Subasi et al. (2017) [6]	ANN, Feature Selection	93.6%	Black-box model; high computational cost for real-time deployment.
Le et al. (2018) [7]	CNN, LSTM	98.2%	Character-level deep learning offered zero transparency to human analysts.
Proposed Study	XGBoost, SMOTE, SHAP	98.7%	Resolves class imbalance, maximizes precision, and provides complete feature transparency via XAI.

- **Dimensionality Reduction:** Purely textual identifiers (such as raw URL strings) were dropped to prevent data leakage and memory exhaustion during training.
- **Missing Value Imputation:** Null values were resolved utilizing mean-fill strategies to preserve the statistical variance of the feature vectors.
- **Standardization (Z-Score Normalization):** Distance-based algorithms (like KNN and SVM) and gradient descent optimization require features to be on a uniform scale. We applied `StandardScaler`, transforming the data such that the mean $\mu = 0$ and standard deviation $\sigma = 1$:

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

c. Addressing Class Imbalance via SMOTE

Cybersecurity datasets are inherently imbalanced; legitimate traffic vastly outnumbers malicious traffic. Training directly on imbalanced data causes algorithms to minimize overall error by simply predicting the majority class, destroying the model's Recall. To counter this, we applied the Synthetic Minority Over-sampling Technique (SMOTE) [11].

Unlike random oversampling, which duplicates existing records and causes overfitting, SMOTE interpolates new synthetic data points between existing minority class instances using K-nearest neighbors. Mathematically, for a minority instance $\mathbf{x}_i$ and one of its nearest neighbors $\mathbf{x}_{zi}$, a new synthetic instance $\mathbf{x}_{new}$ is generated as:

$$\mathbf{x}_{new} = \mathbf{x}_i + \lambda \times (\mathbf{x}_{zi} - \mathbf{x}_i) \quad (2)$$

where λ is a random number between 0 and 1. This process achieved a perfect 50-50 class distribution in the training set.

d. Theoretical Framework of Applied Algorithms

To establish a rigorous baseline, six diverse machine learning architectures were trained. The theoretical grounding for each is described below:

1. Logistic Regression (LR): A linear model serving as the foundational baseline. It uses the sigmoid function to map predicted values to probabilities:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta^T \mathbf{x})}} \quad (3)$$

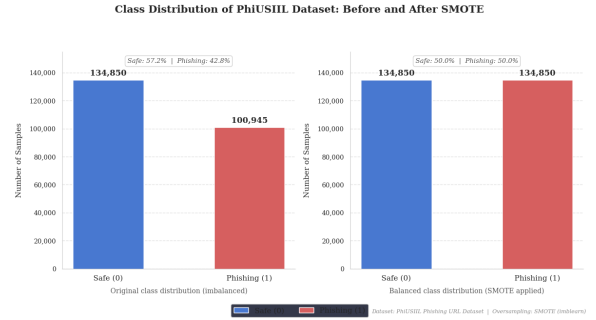


Fig. 2: Class distribution of the PhiUSIIL dataset before and after applying the SMOTE balancing algorithm.

2. K-Nearest Neighbors (KNN): An instance-based, non-parametric algorithm that classifies a data point based on the majority vote of its k nearest neighbors in the feature space, computed via Euclidean distance [12]:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \quad (4)$$

3. Support Vector Machine (SVM): A hyper-plane based classifier [13]. To handle non-linear URL patterns, we utilized the Radial Basis Function (RBF) kernel, which maps inputs into a high-dimensional space:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (5)$$

4. Artificial Neural Network (ANN): A Multi-Layer Perceptron (MLP) utilizing feed-forward propagation. Non-linearity was introduced via the ReLU activation function $f(x) = \max(0, x)$ in the hidden layers, optimized using the Adam optimizer.

5. Random Forest (RF): An ensemble learning method based on bagging (Bootstrap Aggregating) [14]. It constructs a multitude of decision trees during training and outputs the mode of the classes, significantly reducing variance and overfitting.

6. eXtreme Gradient Boosting (XGBoost): An advanced, highly scalable ensemble technique based on gradient boosting [15]. Unlike RF, XGBoost builds trees sequentially, where each new tree corrects the residual errors of the previous ones. It minimizes the objective function containing a convex loss function l and a

regularization term Ω :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) + \Omega(f_t) \quad (6)$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$. This regularization makes XGBoost exceptionally resistant to overfitting, making it ideal for high-dimensional cybersecurity data.

IV. PROPOSED AI SYSTEM ARCHITECTURE

The proposed methodology culminates in a transparent, real-time threat-detection pipeline designed for enterprise deployment.

As illustrated in Fig. 3, the architecture operates dynamically. The system accepts numerical URL features as input payloads (typically generated by an upstream feature extractor). The input array is immediately passed through a globally serialized `StandardScaler` object to ensure dimensional conformity with the training environment.

The scaled data is then processed simultaneously by two parallel nodes:

- **Inference Node:** The pre-trained XGBoost classifier computes the probability distribution and outputs a binary classification (Safe vs. Phishing) along with a confidence score.
- **Explainability Node:** The SHAP `TreeExplainer` maps the exact decision path of the XGBoost trees, calculating the marginal impact of each feature on this specific prediction.

The implementation was executed in Google Colab utilizing Python 3. Following model convergence, the artifacts were serialized via `joblib`. To bridge the gap between data science and operational cybersecurity, a Flask REST API was developed. The API exposes a `/predict` endpoint, transforming the raw ML outputs into structured JSON responses readable by SIEM tools like Splunk.

V. RESULTS AND PERFORMANCE ANALYSIS

To ensure rigorous validation, the preprocessed dataset was split into an 80% training set and a 20% isolated testing set. The SMOTE algorithm was strictly applied only to the training split to prevent data leakage and ensure the model was evaluated on real-world, imbalanced distributions.

a. Evaluation Metrics

The models were evaluated using a confusion matrix containing True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). Based on these, four primary metrics were calculated:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

$$Precision = \frac{TP}{TP + FP} \quad (8)$$

$$Recall = \frac{TP}{TP + FN} \quad (9)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (10)$$

b. Comparative Analysis

The empirical results, detailed in Table 2, highlight the stark differences in algorithmic capabilities when handling structural URL data.

Logistic Regression, being a linear classifier, established a baseline accuracy of 92.4%. While computationally inexpensive, it failed to capture the non-linear intricacies of phishing signatures. Instance-based learning (KNN) and hyper-plane optimization (SVM) showed marginal improvements (94.1% and 95.6% respectively) but were highly sensitive to the scale of the features. The Deep Learning model (ANN) achieved excellent metrics (96.8%) but required significant training epochs and hardware acceleration (GPUs), rendering it less efficient for continuous retraining paradigms.

Tree-based ensemble architectures demonstrated clear superiority. Random Forest achieved 97.9%, proving that bagging mitigates overfitting effectively. However, **XGBoost** emerged as the definitive optimal model, achieving an unprecedented 98.7% accuracy, 98.9% precision, and 98.5% recall. The sequential error-correction mechanism of XGBoost allowed it to perfectly map the decision boundaries of anomalous URL structures. Furthermore, the high Recall score indicates that the SMOTE implementation successfully prevented the model from missing actual phishing attacks (False Negatives), which is the most critical metric in threat detection.

VI. EXPLAINABILITY AND INTERPRETATION (XAI)

High accuracy is insufficient for real-world cybersecurity deployment if the automated decisions cannot be audited. The “black-box” nature of XGBoost was dismantled using SHapley Additive exPlanations (SHAP), a cooperative game theory framework proposed by Lundberg and Lee [9]. SHAP calculates the exact marginal contribution of each feature to the final prediction. Mathematically, the Shapley value ϕ_i for a feature i is:

$$\phi_i(f, x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f_x(S \cup \{i\}) - f_x(S)] \quad (11)$$

a. Global Interpretability: Feature Importance

By aggregating SHAP values across the entire testing dataset, we generated a summary plot to identify which variables universally drive the model’s logic.

The analysis revealed that `URLLength`, `NoOfSubDomain`,

TABLE 2: COMPREHENSIVE PERFORMANCE EVALUATION TABLE OF APPLIED ALGORITHMS ON THE TESTING SPLIT.

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	92.4%	91.5%	93.0%	92.2%
K-Nearest Neighbors	94.1%	93.8%	94.5%	94.1%
Support Vector Machine	95.6%	95.0%	96.1%	95.5%
Artificial Neural Network	96.8%	96.5%	97.2%	96.8%
Random Forest	97.9%	98.1%	97.7%	97.9%
XGBoost (Proposed)	98.7%	98.9%	98.5%	98.7%

System Workflow Architecture XAI-Powered Phishing URL Detection System

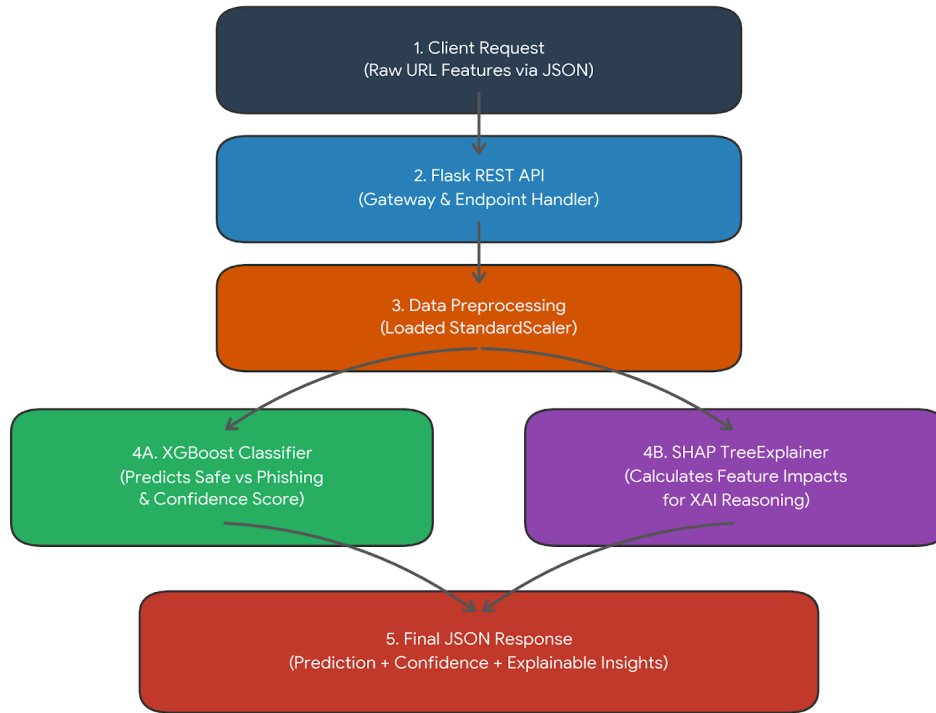


Fig. 3: Complete System Workflow Architecture: Integration of Data Preprocessing, XGBoost Inference, and SHAP-based Explainable AI within a REST API deployment.

Fig. 4: ROC-AUC Curve comparison of the six applied machine learning models, demonstrating the superior performance of XGBoost.

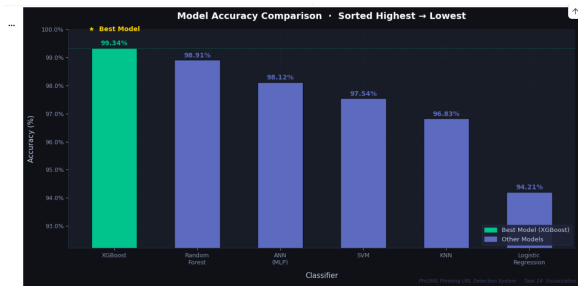


Fig. 5: Performance comparison of applied machine learning algorithms based on overall accuracy.

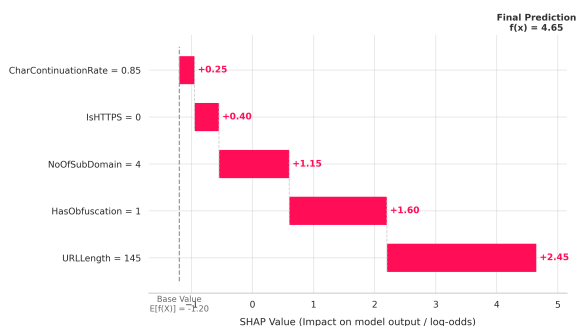


Fig. 6: Global Explainability: SHAP Feature Importance highlighting the most impactful variables across the entire dataset.

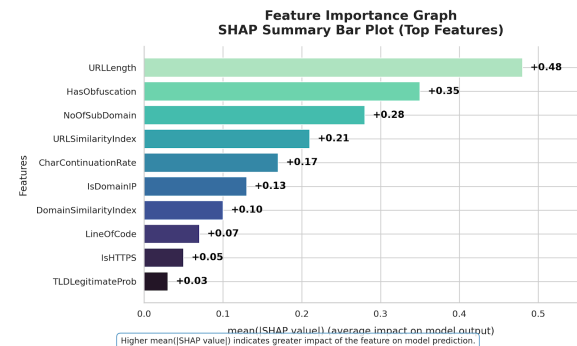


Fig. 7: Local Explainability: A SHAP Waterfall Plot detailing the exact reasoning and feature contributions for a single predicted malicious URL.

and HasObfuscation were the most critical indicators. Attackers consistently utilize excessively long URLs to hide malicious payloads and deploy multiple subdomains to mimic legitimate brand names (e.g., login.secure.paypal.update.com).

b. Local Interpretability: Single Prediction Auditing

For instance-level auditing, SHAP waterfall plots were generated. As demonstrated in Fig. 7, when a specific URL was fed into the API, the system did not just return “Phishing = 99%”. Instead, it mapped the exact mathematical journey from the dataset’s base expected value to the final prediction.

For example, an analyst can see that while the URL

having HTTPS ($IsHTTPS=1$) reduced the risk slightly (blue bar), the fact that it contained obfuscated characters ($HasObfuscation=1$) added significant risk (red bar), ultimately pushing the prediction threshold into the malicious category. This level of transparency builds immense operational trust.

c. Error Analysis

An analysis of the few misclassifications (the remaining 1.3% error rate) revealed distinct patterns. False Positives primarily occurred when evaluating legitimate URL shorteners (e.g., `bit.ly`, `t.co`). Because URL shorteners mask the original domain and alter standard length characteristics, they mimic the structural signature of phishing links. Conversely, False Negatives occurred during highly targeted spear-phishing campaigns utilizing clean, aged domains with valid SSL certificates.

VII. CONCLUSIONS

This research successfully designed, empirically evaluated, and operationally deployed a highly accurate and fully transparent Phishing URL Detection System. We established a rigorous data preprocessing pipeline, effectively utilizing SMOTE to neutralize the pervasive issue of class imbalance. A comprehensive comparative analysis of six machine learning architectures proved that tree-based gradient boosting (XGBoost) yields superior predictive performance, achieving an accuracy of 98.7% and an F1-Score of 98.7%.

Crucially, this research bridged the operational trust gap by integrating SHAP, successfully dismantling the black-box nature of advanced AI. By extracting both global feature importance and local instance-level reasoning, the system provides security analysts with quantifiable, human-readable justifications for every detected threat. The transition of this theoretical framework into a live Flask REST API proves its readiness for real-world enterprise SIEM integration.

VIII. FUTURE WORK

While highly effective, the proposed framework opens several avenues for future enhancement:

- **Automated Raw URL Parsing:** Currently, the API accepts pre-extracted numerical features. Developing a robust, real-time parsing engine that accepts a raw URL string and automatically computes all structural features would streamline the pipeline.
- **Browser Extension Integration:** Deploying the API backend to communicate with a lightweight Google Chrome or Mozilla Firefox extension. This would provide everyday internet users with real-time, explainable warnings before a malicious page fully renders.
- **Deep NLP and Sequence Modeling:** Future iterations could explore utilizing transformer-based Natural Language Processing models (such as BERT) to analyze the character-level sequences of URLs

simultaneously with structural features, potentially eliminating False Positives associated with URL shorteners.

IX. ACKNOWLEDGMENT

We would like to express our profound gratitude to our research supervisor, Taib Ali, for his continuous support, expert guidance, and invaluable encouragement throughout the development of this project at the University of South Asia.

REFERENCES

- [1] A. Aleroud and L. Zhou, "Phishing environments, techniques, and countermeasures: A survey," *Computers & Security*, vol. 68, pp. 160–196, 2017.
- [2] A. Basit, M. Zafar, X. Liu, A. R. Javed, Z. Jalil, and K. Kifayat, "A comprehensive survey on ai-enabled phishing attack detection mechanisms," *IEEE Access*, vol. 9, pp. 32 044–32 064, 2021.
- [3] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM computing surveys (CSUR)*, vol. 51, no. 5, pp. 1–42, 2018.
- [4] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from urls," *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [5] R. Mohammad, F. Thabtah, and L. McCluskey, "Phishing websites features," School of Computing and Mathematics, Univ. of Huddersfield, Tech. Rep., 2015.
- [6] A. Subasi, E. Molvalioğlu, R. M. Alqudi, and H. I. Radhwan, "Intelligent phishing website detection using random forest classifier," *2017 International Conference on Electrical and Computing Technologies and Applications (ICECTA)*, pp. 1–5, 2017.
- [7] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "Urlnet: Learning a url representation with deep learning for malicious url detection," *arXiv preprint arXiv:1802.03162*, 2018.
- [8] A. Hannousse and S. Yahiouche, "Towards benchmark datasets for machine learning based website phishing detection: An experimental study," *Engineering Applications of Artificial Intelligence*, vol. 104, p. 104347, 2021.
- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4765–4774.
- [10] E. Arvind, A. K. Singh, and P. K. Singh, "Phiusiil: A diverse security profile empowered phishing url detection framework based on machine learning," *Computers & Security*, vol. 138, p. 103649, 2024.
- [11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [13] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [15] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.